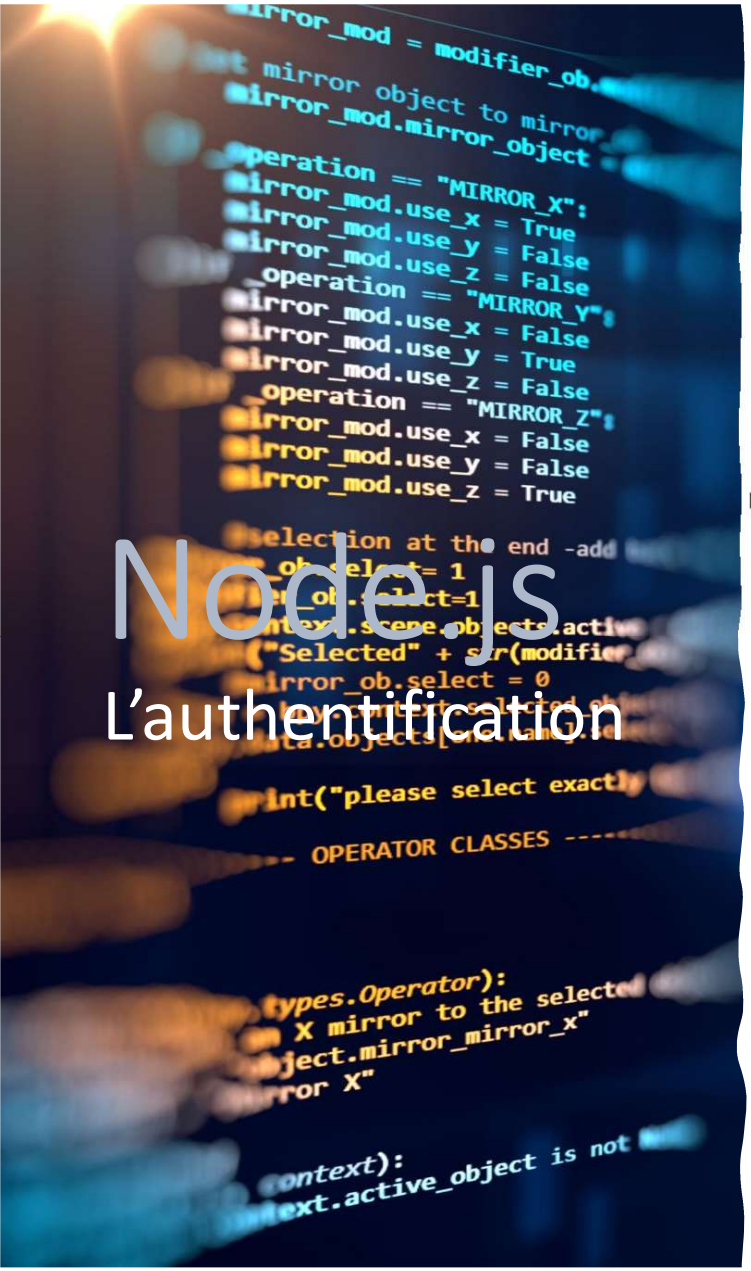
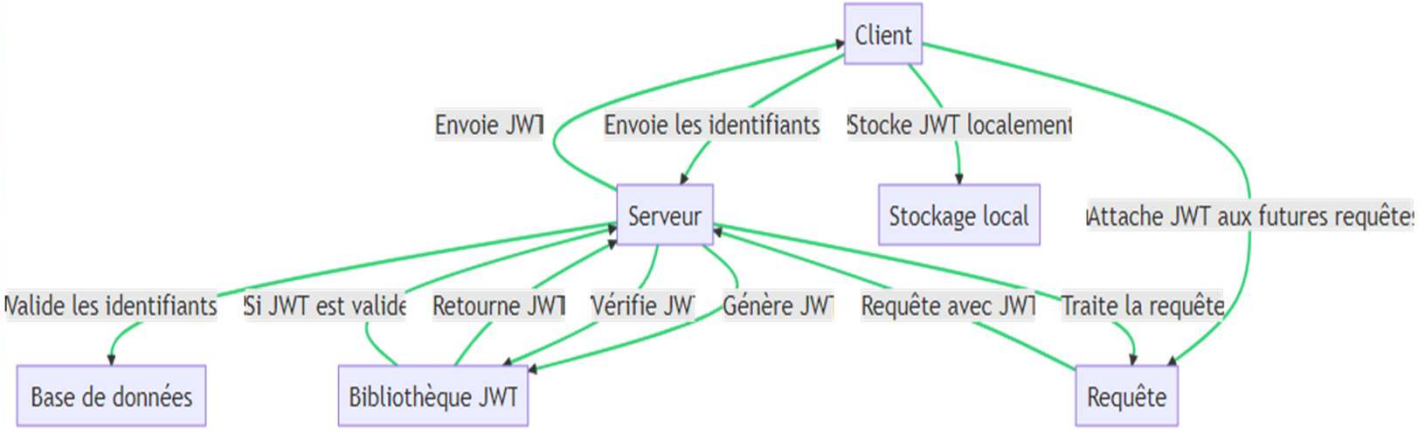
Node.js

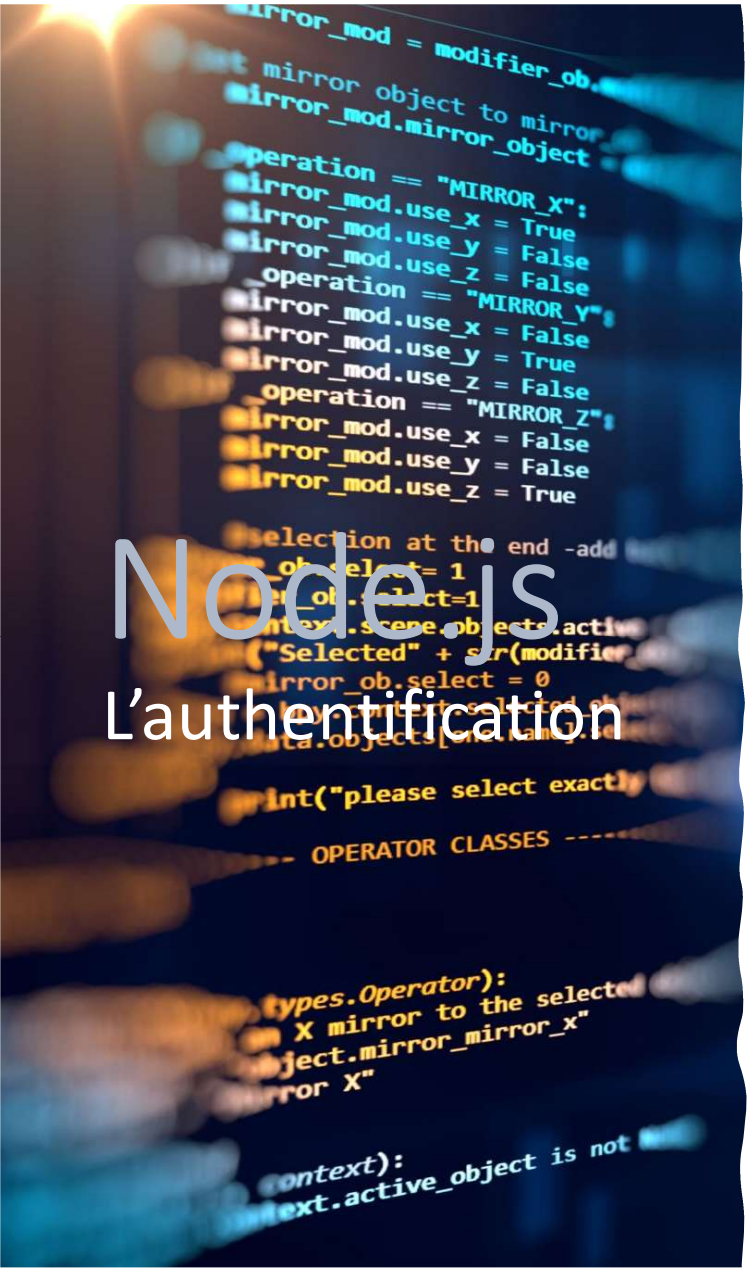
L’authentification



L'authentification JWT en node.js



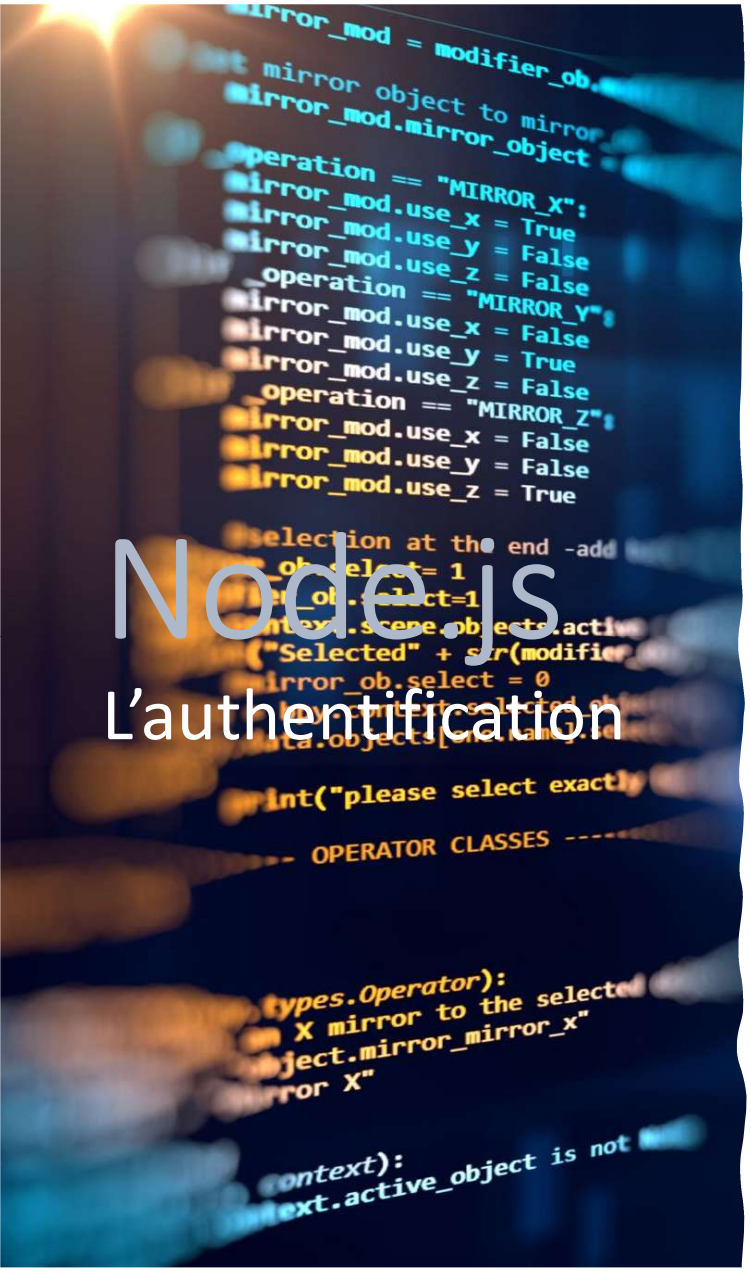
1. Le client envoie ses identifiants au serveur.
2. Le serveur valide les identifiants avec la base de données.
3. Si les identifiants sont valides, le serveur génère un JWT (Token Web JSON) à l'aide d'une bibliothèque JWT.
4. Le serveur renvoie le JWT au client.
5. Le client stocke le JWT localement (généralement dans le stockage local).
6. Pour les futures requêtes, le client attache le JWT à l'en-tête d'autorisation.
7. Le serveur vérifie le JWT. Si le JWT est valide, le serveur traite la requête.



L'authentification JWT en node.js

3 exigences:

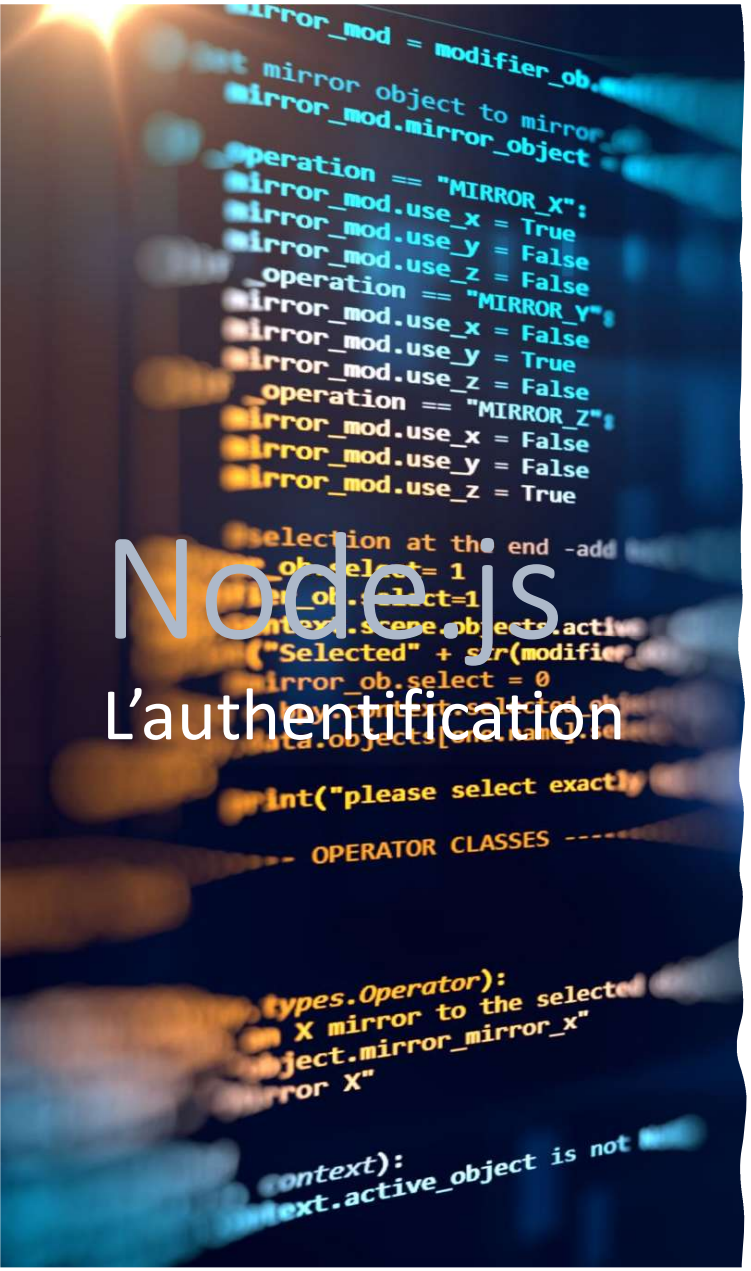
1. Point de terminaison de connexion
2. Encrypter le mot de passe
3. Sécuriser l'échange des données



Créer un modèle pour l'utilisateur

./src/models/authUser.js

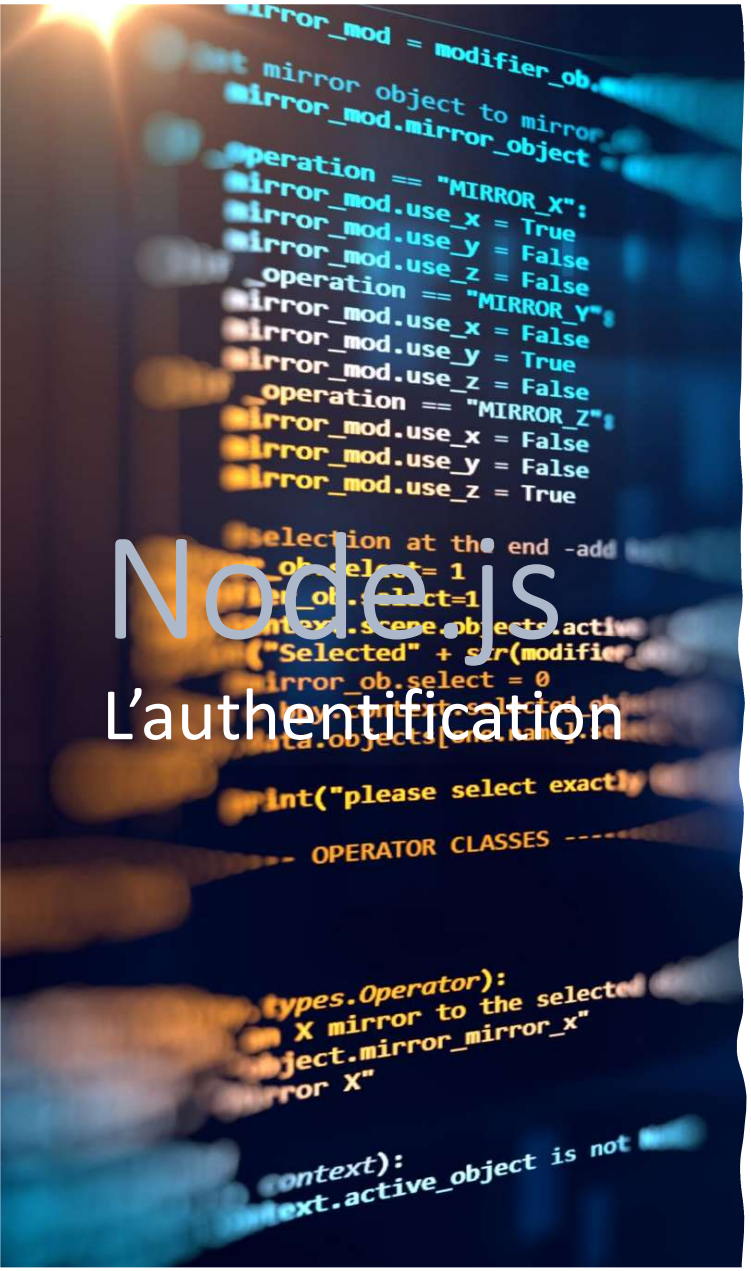
```
const { DataTypes } = require('sequelize');
const { sequelize } = require('../db/sequelize');

const AuthUser = sequelize.define('AuthUser', {
      id: {
        type: DataTypes.INTEGER,
        primaryKey: true,
        autoIncrement: true
      },
      username: {
        type: DataTypes.STRING,
        unique: {                          // contrainte d'unicité
            msg: 'Le nom existe déjà. Veuillez en choisir un autre.'
        }
      },
      password: {
        type: DataTypes.STRING
      }
    });

  module.exports = AuthUser;
```



Créer un modèle pour l'utilisateur

./src/models/authUser.js

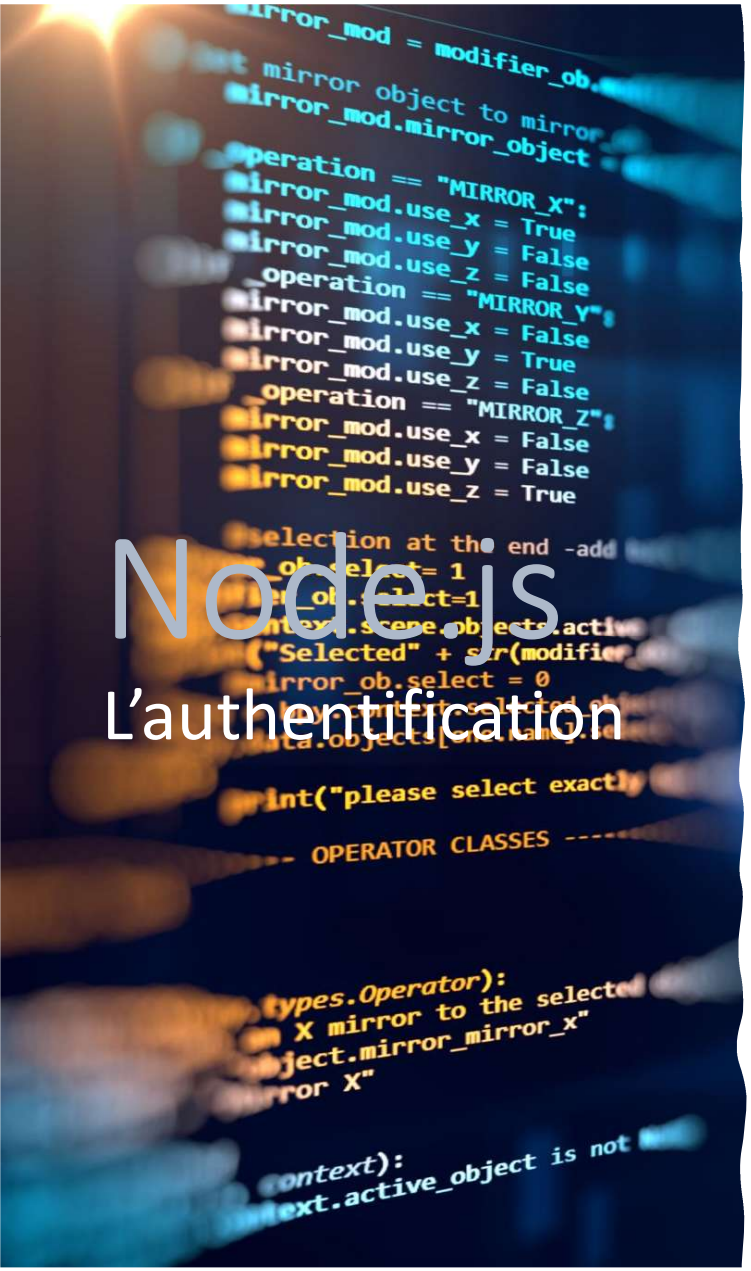
```
const { DataTypes } = require('sequelize');
const { sequelize } = require('../db/sequelize');

const AuthUser = sequelize.define('AuthUser', {
      id: {
        type: DataTypes.INTEGER,
        primaryKey: true,
        autoIncrement: true
      },
      username: {
        type: DataTypes.STRING,
        unique: {                          // contrainte d'unicité
            msg: 'Le nom existe déjà. Veuillez en choisir un autre.'
        }
      },
      password: {
        type: DataTypes.STRING
      }
    });

  module.exports = AuthUser;
```



Créer un utilisateur pour les tests

./src/models/initDb.js

```
// ...
const bcrypt = require('bcrypt');
const AuthUser = require('../models/authUser’);
// ...

await AuthUser.sync({ force: true });

// ...

 // charger un userAuth
    bcrypt.hash('fouad', 10)
    .then(hash => {
      return AuthUser.create({
        username: 'fouad',
        password: hash
      });
    })
    .then(user => {
      console.log(user.toJSON());
    });
```



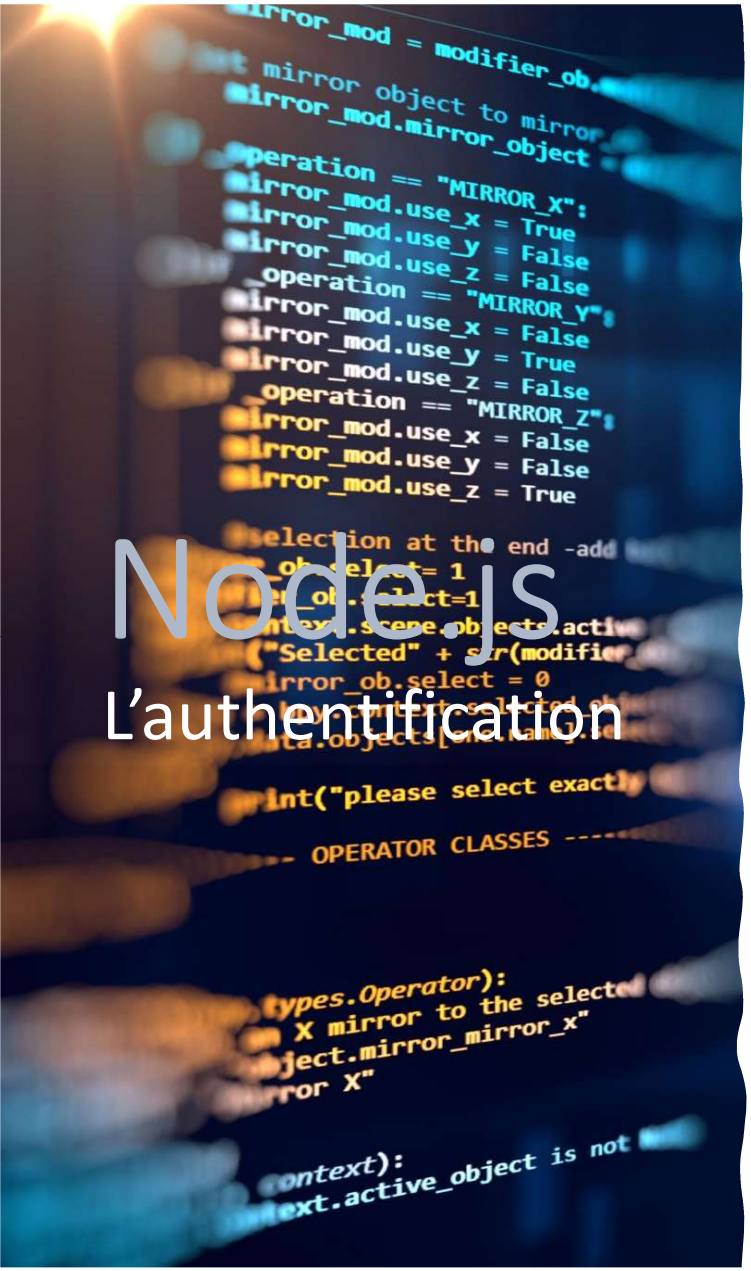
bcrypt.hash()

La méthode bcrypt.hash() est une fonction de hachage fournie par la bibliothèque bcrypt dans Node.js. Elle est utilisée pour transformer une donnée (généralement un mot de passe) en une autre forme, appelée "hash", qui est une chaîne de caractères de longueur fixe.

1. **Input** : La méthode bcrypt.hash() prend deux arguments principaux : le mot de passe à hacher et le nombre de tours (aussi appelé "salt round"). Le nombre de tours est utilisé pour générer le "salt", qui est une chaîne de caractères aléatoire ajoutée au mot de passe pour augmenter la sécurité du hachage.

2. **Processus** : La méthode bcrypt.hash() génère d'abord le "salt" en utilisant le nombre de tours fourni. Ensuite, elle combine le "salt" avec le mot de passe en clair pour créer le hash. Le processus de hachage est unidirectionnel, ce qui signifie qu'il est pratiquement impossible de retrouver le mot de passe original à partir du hash.

3. **Output** : La méthode bcrypt.hash() renvoie une promesse qui résout le hash du mot de passe. Ce hash peut ensuite être stocké en toute sécurité dans une base de données.



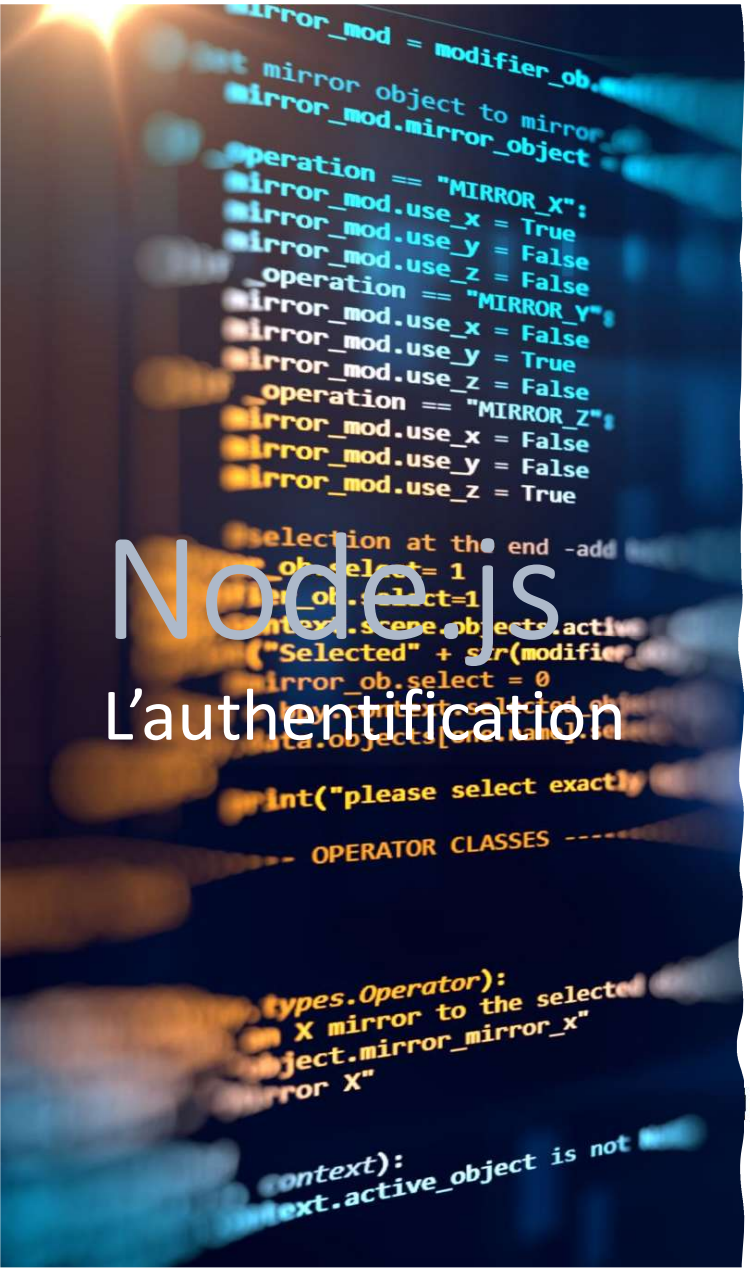
Point de terminaison de connexion

./app.js

```
// ...
app.use('/login', require('./src/routes/loginRoutes'));
// ...
```

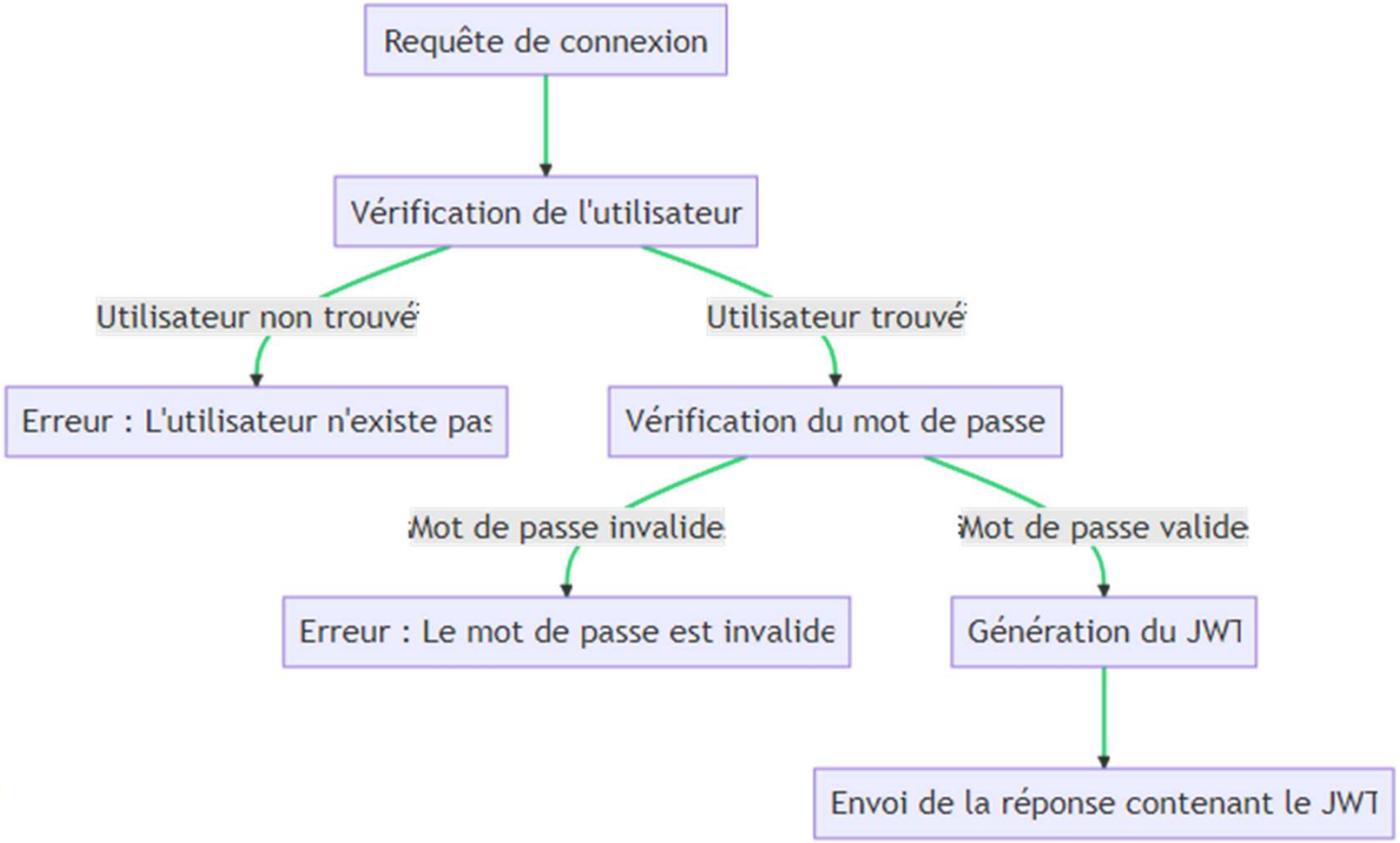
./src/routes/loginRoutes.js

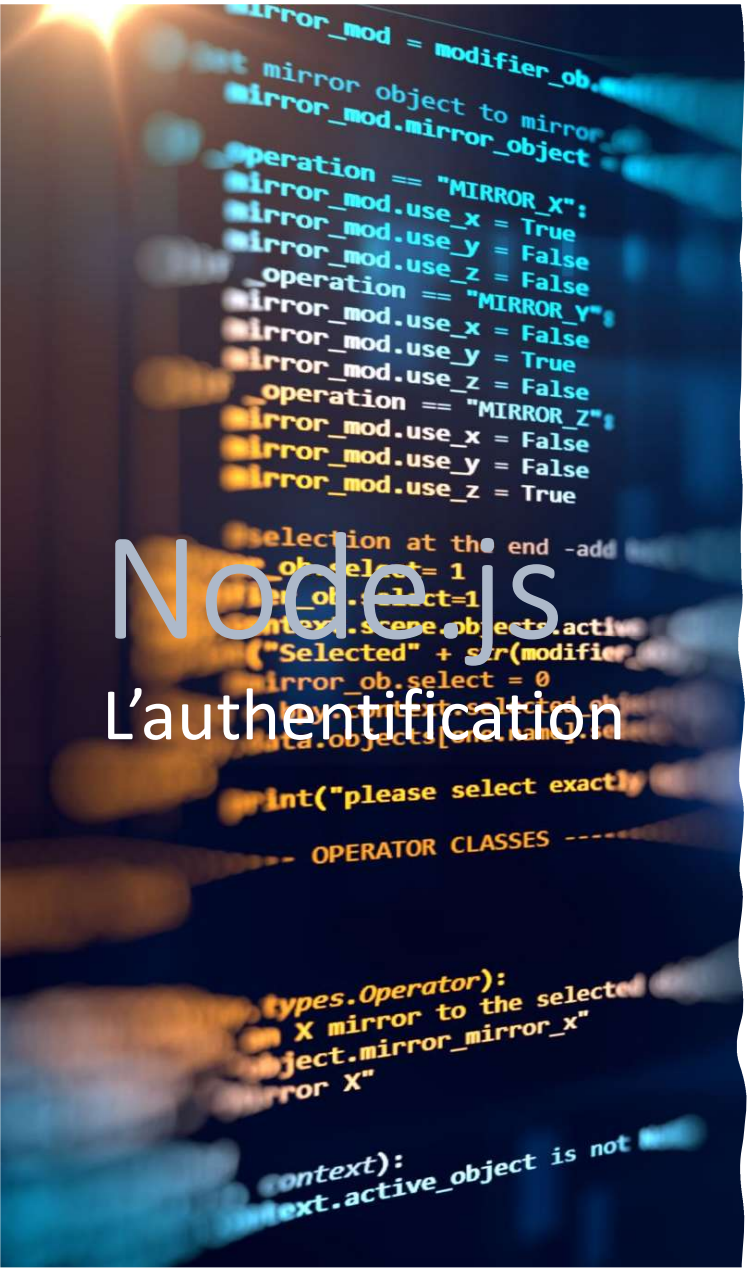
```
const express = require('express');
const router = express.Router();
const loginController = require('../controllers/loginController');
//const bcrypt = require('bcrypt')
router.get('/', loginController.login);


module.exports = router;
```



Point de terminaison de connexion

./src/controllers/loginController.js

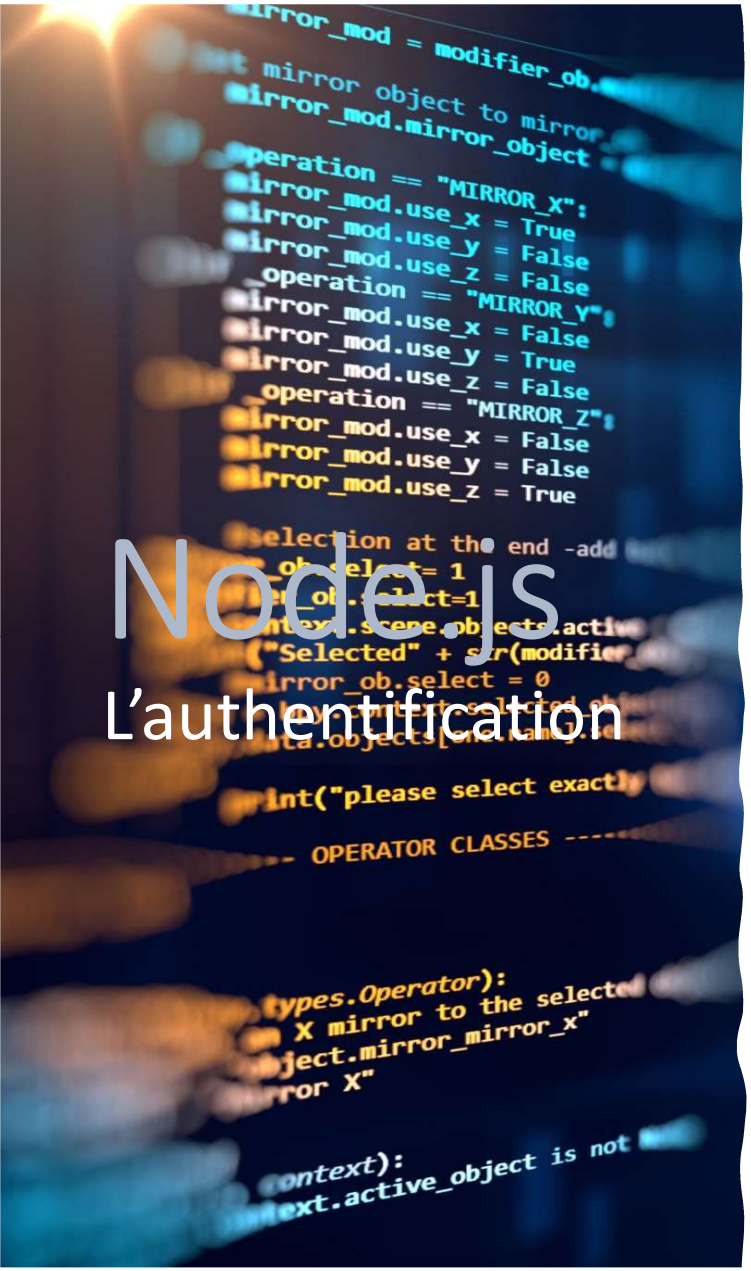




Point de terminaison de connexion

Pour génerer un JWT il faut:

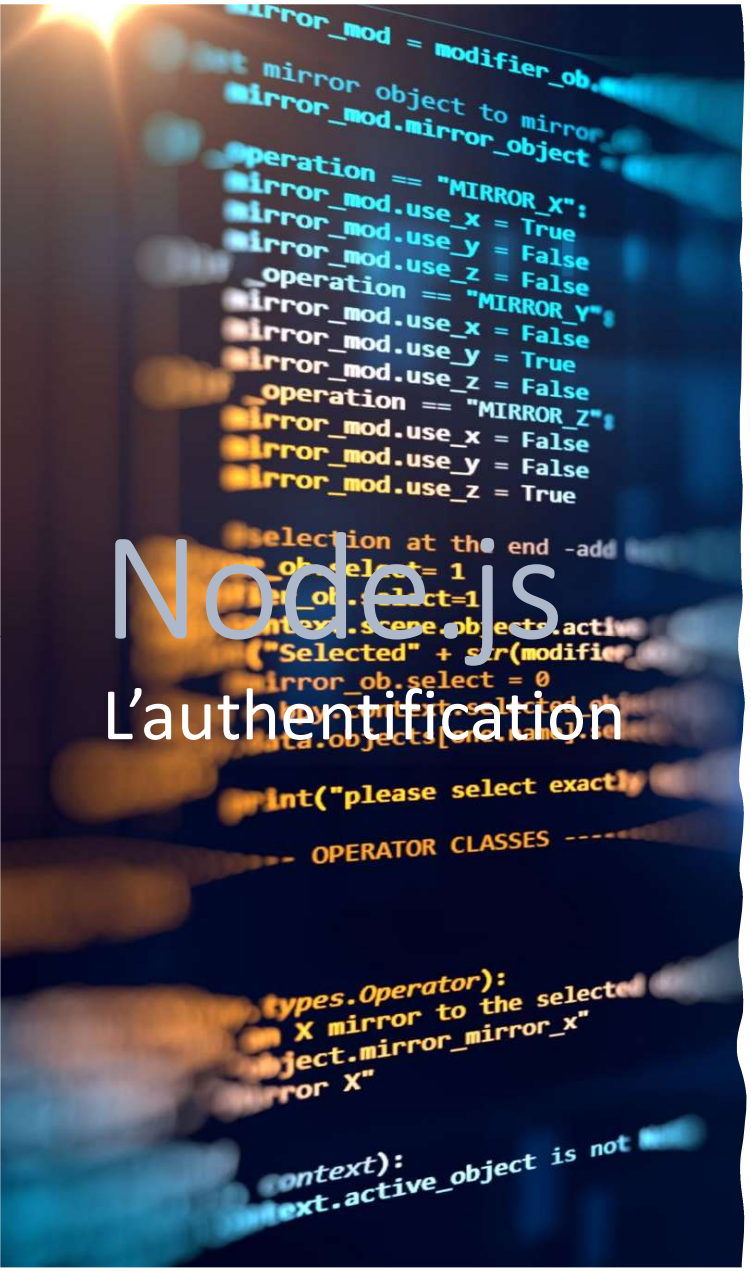
- Les informations de l'utilisateur
- Une clef decrète
- Une durée de validité



Point de terminaison de connexion

./src/auth/private_key.js

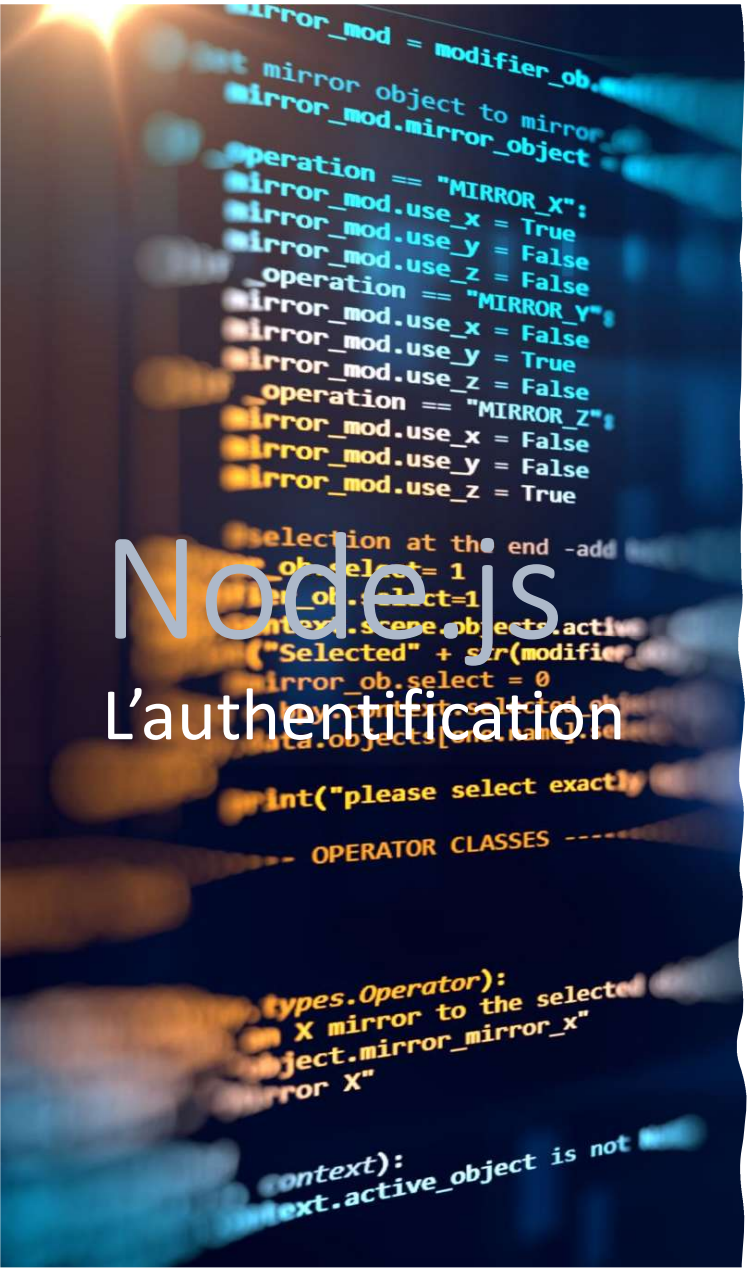
```
module.exports = 'CUSTOM_PRIVATE_KEY'
```



Point de terminaison de connexion

./src/controllers/loginController.js

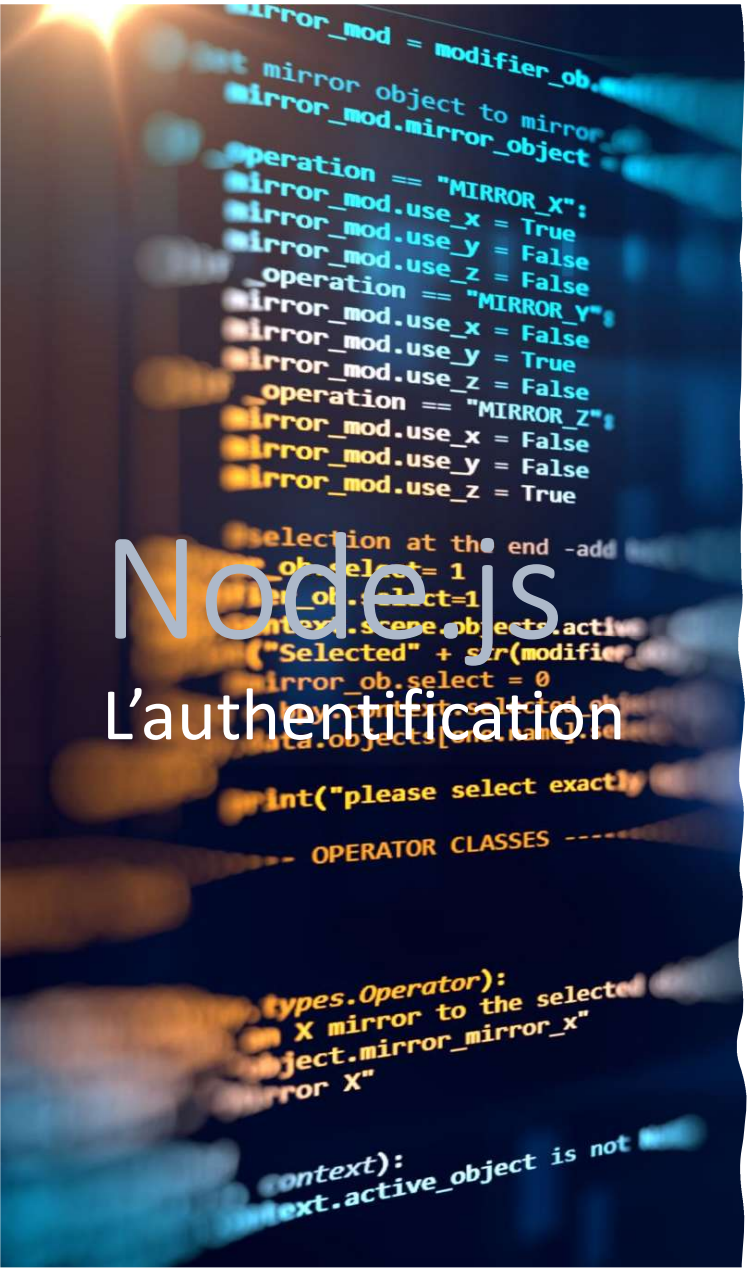
```
const AuthUser = require('../models/authUser');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken’);           // le module pour JWT
const privatekey = require('../auth/private_key')
const loginController = {
    login: async (req, res) => {
            AuthUser.findOne({ where: { username: req.body.username } }).then(user => {
                if (!user) {
                    const message = `L'utilisateur n'existe pas.`;
                    return res.status(404).json({ message })
                }
                bcrypt.compare(req.body.password, user.password).then(isPasswordValid => {
                    if (!isPasswordValid) {
                        const message = `Le mot de passe est inavalide.`;
                        return res.status(404).json({ message })
                    }
                    // JWT
                    const token = jwt.sign(
                        { userId: user.id},
                        privatekey,
                        { expiresIn: '24h' }
                    )
                    const message = `L'utilisateur a été connecté avec succès`;
                    return res.json({ message, data: user, token })
                })
            })
                .catch(error => {
                    const message = `Vous návez pas été conncté. Réessayez dans quelques instants.`;
                    return res.status(404).json({ message, data: error })
                })
    }
}
module.exports = loginController;
```



Middleware d'authentification

./src/auth/auth.js

```
const jwt = require('jsonwebtoken')
const privateKey = require('../auth/private_key')

module.exports = (req, res, next) => {
  const authorizationHeader = req.headers.authorization

  if (!authorizationHeader) {
    const message = `Vous n'avez pas fourni de jeton d'authentification. Ajoutez-en un
dans l'en-tête de la requête.`
    return res.status(401).json({ message })
  }

  const token = authorizationHeader.split(' ')[1]
  const decodedToken = jwt.verify(token, privateKey, (error, decodedToken) => {
    if (error) {
      const message = `L'utilisateur n'est pas autorisé à accèder à cette ressource.`
      return res.status(401).json({ message, data: error })
    }

    const userId = decodedToken.userId
    if (req.body.userId && req.body.userId !== userId) {
      const message = `L'identifiant de l'utilisateur est invalide.`
      res.status(401).json({ message })
    } else {
      next()
    }
  })
}
```



Middleware d'authentification

Le middleware auth doit se déclencher pour tous les points de terminaison, afin de vérifier le jeton, excepté pour le point de terminaison de connexion.

./app.js

```
const express = require('express');
const port = 3000;
const app = express();

const initDb =require('./src/db/initDb');
const auth = require('./src/auth/auth');
app.use(express.json());
const cors = require('cors');
app.use(cors);

app.use('/users', auth, require('./src/routes/userRoutes'));
app.use('/books', auth, require('./src/routes/bookRoutes'));
app.use('/prets', auth, require('./src/routes/pretRoutes'));
app.use('/login', require('./src/routes/loginRoutes'));     //
ajouter le point de terminaison

app.listen(port, () => console.log(`Server running on port ${port}`));
```